

Code Assignment

Overview

Thank you for your interest in the Backend Software Engineer position at Rapyuta Robotics. This take-home assignment is designed to assess your ability to design and implement a simple, robust backend tool.

- It should take approximately 2-4 hours to complete, and you have one week from the date you receive this to submit your solution.
- Use Python as the main programming language. Feel free to use other languages / tools to accommodate any specific needs.
- Keep your code clean, organized, and well-written.
- If you have any clarifying questions, feel free to ask the recruiter via email
- Good luck!

Requirements

Your task is to build a **location and inventory management tool**.

- A **location** is a place in the warehouse where inventory may be stored, represented by a globally unique name (e.g., "LA" for Location A).
- An **inventory** is a pair consisting of an item identifier (a string, e.g., "IA" for Item A) and a positive quantity (>0).
- Locations and item IDs are alphanumeric strings (no spaces), while quantities are positive integers.

The tool should support the following operations:

1. **Registering and deregistering locations:**
 - a. Register a new location by its unique name.
 - b. Registration of an existing location should fail.
 - c. Deregistration should only succeed if the location exists and has no inventory. Otherwise, it should fail.
2. **Incrementing inventory** in a registered location: Add a specified quantity of an item to a location. If the item doesn't exist in the location, start with the given quantity.
3. **Decrementing inventory** in a registered location: Subtract a specified quantity of an item from a location. Fail if the location doesn't have enough of that item (quantity would go <0).
4. **Transferring inventory** between registered locations: Move a specified quantity of an item from one location to another. Fail if the source doesn't have enough quantity or if either location doesn't exist.

5. **Observing inventories** in a registered location: Retrieve the list of items and their quantities in a specific location.

Take the following into consideration:

- **Input/Output:** Implement this as a command-line tool that reads commands from standard input (stdin) and writes responses to standard output (stdout). Each command is a single line, and the tool should process commands one by one.
- **State Persistence:** The state of the warehouse (locations and their inventories) should always be stored in a file (format up to you, e.g., JSON, CSV, or another simple format; SQLite is also acceptable).

The program should load the state from the file on startup (if the file exists) and save the state to the file after each successful operation that modifies it. It should be possible to work with multiple instances of the tool running concurrently (e.g., multiple processes accessing the same file).

- No need for authentication or external dependencies (use only Python standard library).
- The persistence file should be hardcoded (e.g., "warehouse_state.json") or configurable via a command-line argument (your choice, explain in README).

The deliverable should be a Zip file containing the following:

- Source code.
- A short README.md file explaining:
 - Your design decisions (e.g., data structures chosen, import/export format and handling, and why).
 - Instructions on how to run the code.

Command Details and Examples

The tool should handle the following command types:

1. **LOCATION REGISTER <LOCATION_ID>**
 - a. Registers a new location.
 - b. Fails if the location already exists.
2. **LOCATION UNREGISTER <LOCATION_ID>**
 - a. Deregisters a location.
 - b. Fails if the location doesn't exist or has any inventory.
3. **INVENTORY INCREMENT <LOCATION_ID> <ITEM_ID> <QUANTITY>**
 - a. Adds the quantity to the item in the location.
 - b. If the item doesn't exist, create it with that quantity.
 - c. Fails if the location doesn't exist.
4. **INVENTORY DECREMENT <LOCATION_ID> <ITEM_ID> <QUANTITY>**
 - a. Subtracts the quantity from the item in the location.
 - b. Fails if the location doesn't exist, the item doesn't exist, or there's insufficient quantity.
5. **INVENTORY TRANSFER <SRC_LOCATION_ID> <DEST_LOCATION_ID> <ITEM_ID> <QUANTITY>**
 - a. Moves the quantity of the item from source to destination.

- b. Adds to target if the item doesn't exist there.
 - c. Fails if either location doesn't exist, or source has insufficient quantity.
- 6. INVENTORY OBSERVE <LOCATION_ID>**
- a. Outputs the inventories in the format: "ITEM " for each item, one per line, sorted alphabetically by item_id. If no items, output "EMPTY".
 - b. Fails if the location doesn't exist.

Example Input/Output

Assume the tool is run and processes the following commands sequentially:

```
LOCATION REGISTER LA
> OK

LOCATION REGISTER LB
> OK

LOCATION REGISTER LA
> ERR: Location 'LA' already exists

INVENTORY INCREMENT LA IA 5
> OK

INVENTORY DECREMENT LA IA 3
> OK

INVENTORY DECREMENT LA IA 3
> ERR: Insufficient quantity of item IA in location LA (has 2)

INVENTORY TRANSFER LA LB IA 2
> OK

LOCATION UNREGISTER LA
> OK

LOCATION UNREGISTER LC
> ERR: Location 'LC' does not exist

LOCATION UNREGISTER LB
> ERR: Location 'LB' has inventories

INVENTORY OBSERVE LB
> ITEM IA 2

INVENTORY OBSERVE LA
> ERR: Location 'LA' does not exist

LOCATION REGISTER LA
> OK

INVENTORY OBSERVE LA
> EMPTY
```